

Note that by default, the text is aligned above and to the left of the specified coordinates; here the “.” at the beginning of each string will approximately mark the given coordinate location.

The `transData` coordinates give the usual data coordinates associated with the x- and y-axis labels. The `transAxes` coordinates give the location from the bottom-left corner of the axes (here the white box) as a fraction of the axes size.

The `transfigure` coordinates are similar, but specify the position from the bottom left of the figure (here the gray box) as a fraction of the figure size.

Notice now that if we change the axes limits, it is only the `transData` coordinates that will be affected, while the others remain stationary.

Arrows and Annotation

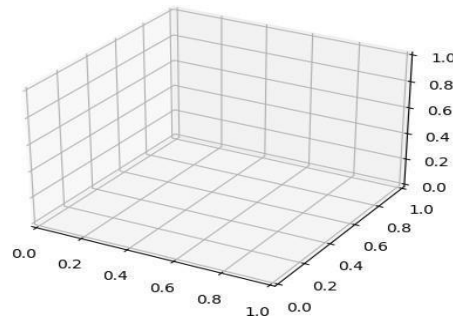
- Along with tick marks and text, another useful annotation mark is the simple arrow.
- Drawing arrows in Matplotlib is not much harder because there is a `plt.arrow()` function available.
- The arrows it creates are SVG (scalable vector graphics) objects that will be subject to the varying aspect ratio of your plots, and the result is rarely what the user intended.
- The arrow style is controlled through the `arrowprops` dictionary, which has numerous options available.

Three-Dimensional Plotting in Matplotlib

We enable three-dimensional plots by importing the `mplot3d` toolkit, included with the main Matplotlib installation.

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits import mplot3d
fig = plt.figure()
ax = plt.axes(projection='3d')
```

With this 3D axes enabled, we can now plot a variety of three-dimensional plot types.

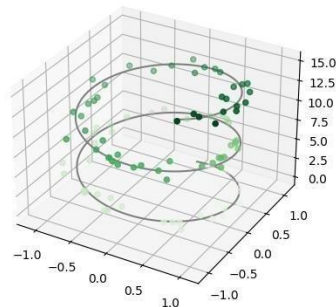


Three-Dimensional Points and Lines

The most basic three-dimensional plot is a line or scatter plot created from sets of (x, y, z) triples.

In analogy with the more common two-dimensional plots discussed earlier, we can create these using the `ax.plot3D` and `ax.scatter3D` functions

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits import mplot3d
ax = plt.axes(projection='3d')
# Data for a three-dimensional line
zline = np.linspace(0, 15, 1000)
xline = np.sin(zline)
yline = np.cos(zline)
ax.plot3D(xline, yline, zline, 'gray')
# Data for three-dimensional scattered points
zdata = 15 * np.random.random(100)
xdata = np.sin(zdata) + 0.1 * np.random.randn(100)
ydata = np.cos(zdata) + 0.1 * np.random.randn(100)
```



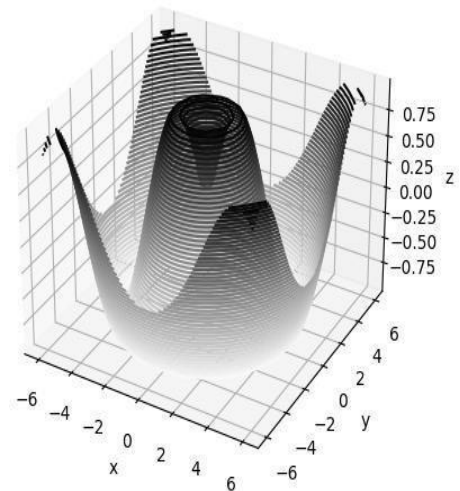
```
ax.scatter3D(xdata, ydata, zdata, c=zdata, cmap='Greens');
plt.show()
```

Notice that by default, the scatter points have their transparency adjusted to give a sense of depth on the page.

Three-Dimensional Contour Plots

- `mplot3d` contains tools to create three-dimensional relief plots using the same inputs.
- Like two-dimensional `ax.contour` plots, `ax.contour3D` requires all the input data to be in the form of two-dimensional regular grids, with the Z data evaluated at each point.
- Here we'll show a three-dimensional contour diagram of a three dimensional sinusoidal function

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits import mplot3d
def f(x, y):
    return np.sin(np.sqrt(x ** 2 + y ** 2))
x = np.linspace(-6, 6, 30)
y = np.linspace(-6, 6, 30)
X, Y = np.meshgrid(x, y)
Z = f(X, Y)
fig = plt.figure()
ax = plt.axes(projection='3d')
ax.contour3D(X, Y, Z, 50, cmap='binary')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
plt.show()
```



Sometimes the default viewing angle is not optimal, in which case we can use the `view_init` method to set the elevation and azimuthal angles.

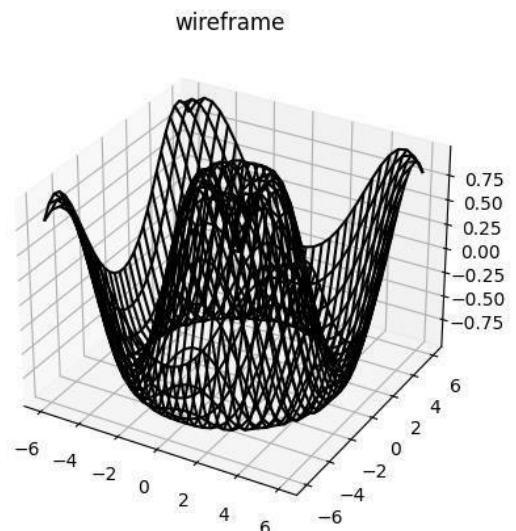
```
ax.view_init(60, 35)
fig
```

Wire frames and Surface Plots

- Two other types of three-dimensional plots that work on gridded data are wireframes and surface plots.
- These take a grid of values and project it onto the specified three-dimensional surface, and can make the resulting three-dimensional forms quite easy to visualize.

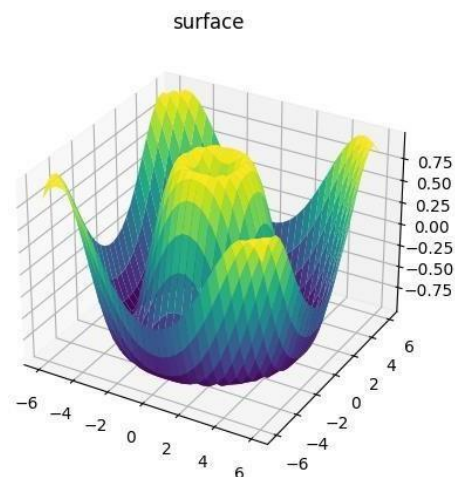
```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits import mplot3d
fig = plt.figure()
ax = plt.axes(projection='3d')
ax.plot_wireframe(X, Y, Z, color='black')
ax.set_title('wireframe');
plt.show()
```

- A surface plot is like a wireframe plot, but each face of the wireframe is a filled polygon.



- Adding a colormap to the filled polygons can aid perception of the topology of the surface being visualized

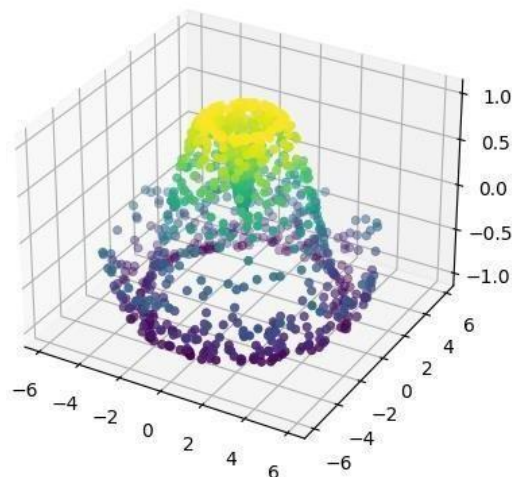
```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits import mplot3d
ax = plt.axes(projection='3d')
ax.plot_surface(X, Y, Z, rstride=1, cstride=1,
cmap='viridis', edgecolor='none')
ax.set_title('surface')
plt.show()
```



Surface Triangulations

- For some applications, the evenly sampled grids required by the preceding routines are overly restrictive and inconvenient.
- In these situations, the triangulation-based plots can be very useful.

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits import mplot3d
theta = 2 * np.pi * np.random.random(1000)
r = 6 * np.random.random(1000)
x = np.ravel(r * np.sin(theta))
y = np.ravel(r * np.cos(theta))
z = f(x, y)
ax = plt.axes(projection='3d')
ax.scatter(x, y, z, c=z, cmap='viridis', linewidth=0.5)
```



Geographic Data with Basemap

- One common type of visualization in data science is that of geographic data.
- Matplotlib's main tool for this type of visualization is the Basemap toolkit, which is one of several Matplotlib toolkits that live under the `mpl_toolkits` namespace.
- Basemap is a useful tool for Python users to have in their virtual toolbelts
- Installation of Basemap. Once you have the Basemap toolkit installed and imported, geographic plots also require the PIL package in Python 2, or the pillow package in Python 3.

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.basemap import Basemap
plt.figure(figsize=(8, 8))
m = Basemap(projection='ortho', resolution=None,
lat_0=50, lon_0=-100)
m.bluemarble(scale=0.5);
```

- Matplotlib axes that understands spherical coordinates and allows us to easily over-plot data on the map

- We'll use an etopo image (which shows topographical features both on land and under the ocean) as the map background

Program to display particular area of the map with latitude and longitude lines

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.basemap import Basemap
from itertools import chain
fig = plt.figure(figsize=(8, 8))
m = Basemap(projection='lcc', resolution=None,
width=8E6, height=8E6,
lat_0=45, lon_0=-100,)
m.etopo(scale=0.5, alpha=0.5)
def draw_map(m, scale=0.2):
    # draw a shaded-relief image
    m.shadedrelief(scale=scale)
    # lats and longs are returned as a dictionary
    lats = m.drawparallels(np.linspace(-90, 90, 13))
    lons = m.drawmeridians(np.linspace(-180, 180, 13))
    # keys contain the plt.Line2D instances
    lat_lines = chain(*(tup[1][0] for tup in lats.items()))
    lon_lines = chain(*(tup[1][0] for tup in lons.items()))
    all_lines = chain(lat_lines, lon_lines)
    # cycle through these lines and set the desired style
    for line in all_lines:
        line.set(linestyle='-', alpha=0.3, color='r')
```



Map Projections

The Basemap package implements several dozen such projections, all referenced by a short format code. Here we'll briefly demonstrate some of the more common ones.

- Cylindrical projections
- Pseudo-cylindrical projections
- Perspective projections
- Conic projections

Cylindrical projection

- The simplest of map projections are cylindrical projections, in which lines of constant latitude and longitude are mapped to horizontal and vertical lines, respectively.
- This type of mapping represents equatorial regions quite well, but results in extreme distortions near the poles.
- The spacing of latitude lines varies between different cylindrical projections, leading to different conservation properties, and different distortion near the poles.
- Other cylindrical projections are the Mercator (projection='merc') and the cylindrical equal-area (projection='cea') projections.
- The additional arguments to Basemap for this view specify the latitude (lat) and longitude (lon) of the lower-left corner (llcrnr) and upper-right corner (urcrnr) for the desired map, in units of degrees.

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.basemap import Basemap
```

```
fig = plt.figure(figsize=(8, 6), edgecolor='w')
m = Basemap(projection='cyl', resolution=None,
llcrnrlat=-90, urcrnrlat=90,
llcrnrlon=-180, urcrnrlon=180,)
draw_map(m)
```



Pseudo-cylindrical projections

- Pseudo-cylindrical projections relax the requirement that meridians (lines of constant longitude) remain vertical; this can give better properties near the poles of the projection.
- The Mollweide projection (projection='moll') is one common example of this, in which all meridians are elliptical arcs
- It is constructed so as to
- preserve area across the map: though there are distortions near the poles, the area of small patches reflects the true area.
- Other pseudo-cylindrical projections are the sinusoidal (projection='sinu') and Robinson (projection='robin') projections.
- The extra arguments to Basemap here refer to the central latitude (lat_0) and longitude (lon_0) for the desired map.

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.basemap import Basemap
fig = plt.figure(figsize=(8, 6), edgecolor='w')
m = Basemap(projection='moll', resolution=None,
lat_0=0, lon_0=0)
draw_map(m)
```



Perspective projections

- Perspective projections are constructed using a particular choice of perspective point, similar to if you photographed the Earth from a particular point in space (a point which, for some projections, technically lies within the Earth!).

- One common example is the orthographic projection (projection='ortho'), which shows one side of the globe as seen from a viewer at a very long distance.
- Thus, it can show only half the globe at a time.
- Other perspective-based projections include the gnomonic projection (projection='gnom') and stereographic projection (projection='stere').
- These are often the most useful for showing small portions of the map.

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.basemap import Basemap
fig = plt.figure(figsize=(8, 8))
m = Basemap(projection='ortho', resolution=None,
lat_0=50, lon_0=0)
draw_map(m);
```



Conic projections

- A conic projection projects the map onto a single cone, which is then unrolled.
- This can lead to very good local properties, but regions far from the focus point of the cone may become very distorted.
- One example of this is the Lambert conformal conic projection (projection='lcc').
- It projects the map onto a cone arranged in such a way that two standard parallels (specified in Basemap by lat_1 and lat_2) have well-represented distances, with scale decreasing between them and increasing outside of them.
- Other useful conic projections are the equidistant conic (projection='eqdc') and the Albers equal-area (projection='aea') projection

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.basemap import Basemap
fig = plt.figure(figsize=(8, 8))
m = Basemap(projection='lcc', resolution=None,
lon_0=0, lat_0=50, lat_1=45, lat_2=55, width=1.6E7, height=1.2E7)
draw_map(m)
```



Drawing a Map Background

The Basemap package contains a range of useful functions for drawing borders of physical features like continents, oceans, lakes, and rivers, as well as political boundaries such as countries and US states and counties.

The following are some of the available drawing functions that you may wish to explore using IPython's help features:

- Physical boundaries and bodies of water

`drawcoastlines()` - Draw continental coast lines

`drawlsmask()` - Draw a mask between the land and sea, for use with projecting images on one or the other

`drawmapboundary()` - Draw the map boundary, including the fill color for oceans

`drawrivers()` - Draw rivers on the map

`fillcontinents()` - Fill the continents with a given color; optionally fill lakes with another color

- Political boundaries

`drawcountries()` - Draw country boundaries

`drawstates()` - Draw US state boundaries

`drawcounties()` - Draw US county boundaries

- Map features

`drawgreatcircle()` - Draw a great circle between two points

`drawparallels()` - Draw lines of constant latitude

`drawmeridians()` - Draw lines of constant longitude

`drawmapscale()` - Draw a linear scale on the map

- Whole-globe images

`bluemarble()` - Project NASA's blue marble image onto the map

`shadedrelief()` - Project a shaded relief image onto the map

`etopo()` - Draw an etopo relief image onto the map

`warpimage()` - Project a user-provided image onto the map

Plotting Data on Maps

- The Basemap toolkit is the ability to over-plot a variety of data onto a map background.
- There are many map-specific functions available as methods of the Basemap instance.

Some of these map-specific methods are:

`contour()/contourf()` - Draw contour lines or filled contours

`imshow()` - Draw an image

`pcolor()/pcolormesh()` - Draw a pseudocolor plot for irregular/regular meshes

`plot()` - Draw lines and/or markers

`scatter()` - Draw points with markers

`quiver()` - Draw vectors

`barbs()` - Draw wind barbs

`drawgreatcircle()` - Draw a great circle

Visualization with Seaborn

The main idea of Seaborn is that it provides high-level commands to create a variety of plot types useful for statistical data exploration, and even some statistical model fitting.

Histograms, KDE, and densities

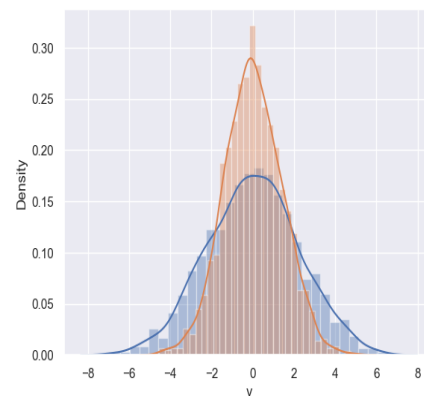
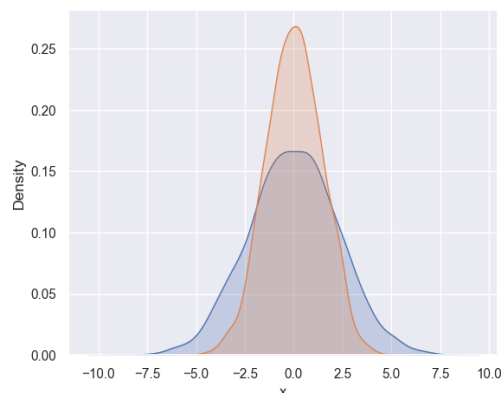
- In statistical data visualization, all you want is to plot histograms and joint distributions of variables. We have seen that this is relatively straightforward in Matplotlib
- Rather than a histogram, we can get a smooth estimate of the distribution using a kernel density estimation, which Seaborn does with `sns.kdeplot`

```
import pandas as pd
import seaborn as sns
data = np.random.multivariate_normal([0, 0], [[5, 2], [2, 2]], size=2000)
data = pd.DataFrame(data, columns=['x', 'y'])
for col in 'xy':
    sns.kdeplot(data[col], shade=True)
```

- Histograms and KDE can be combined using `distplot`

```
sns.distplot(data['x'])
sns.distplot(data['y']);
```

- If we pass the full two-dimensional dataset to `kdeplot`, we will get a two-dimensional visualization of the data.
- We can see the joint distribution and the marginal distributions together using `sns.jointplot`.

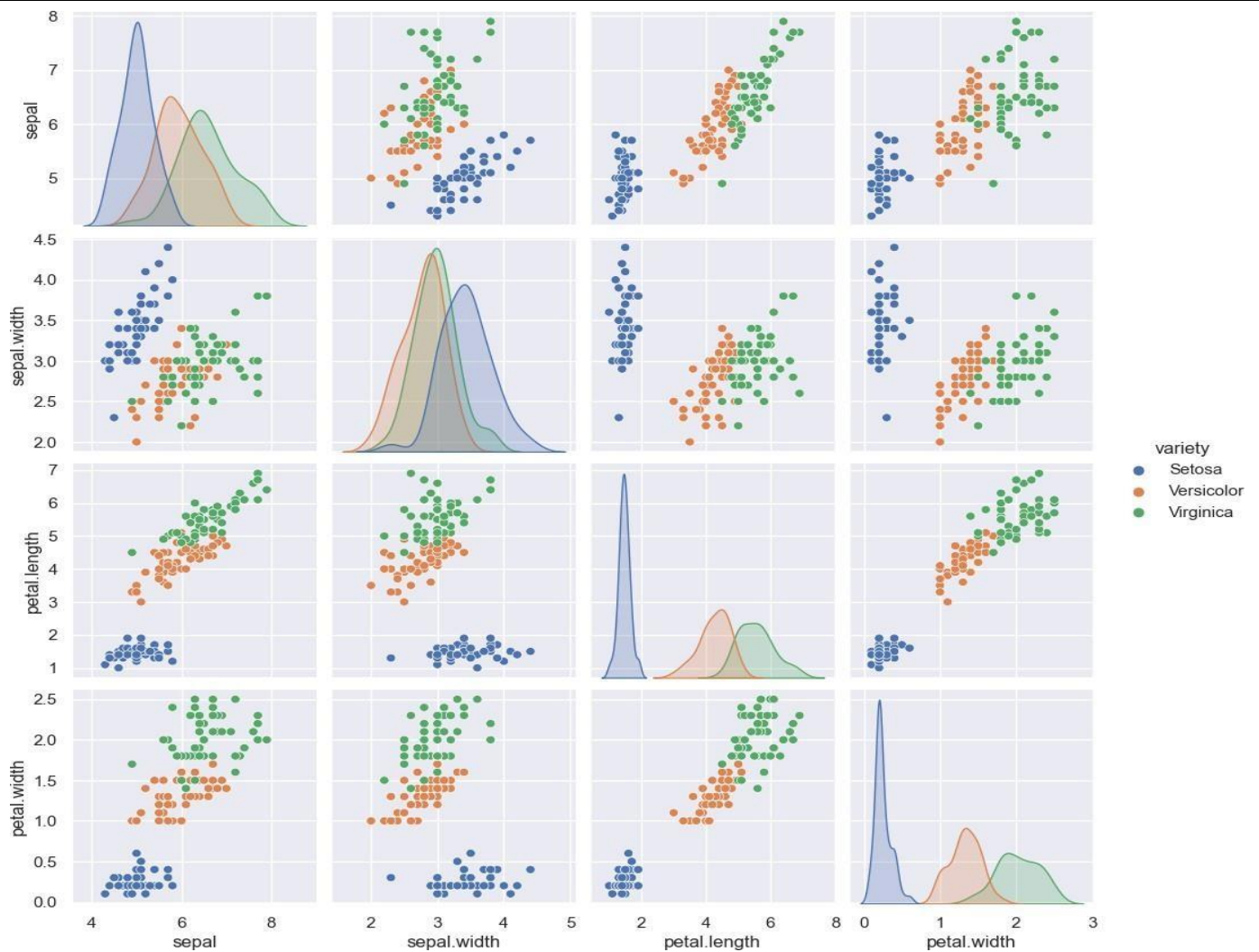


Pair plots

When you generalize joint plots to datasets of larger dimensions, you end up with pair plots. This is very useful for exploring correlations between multidimensional data, when you'd like to plot all pairs of values against each other.

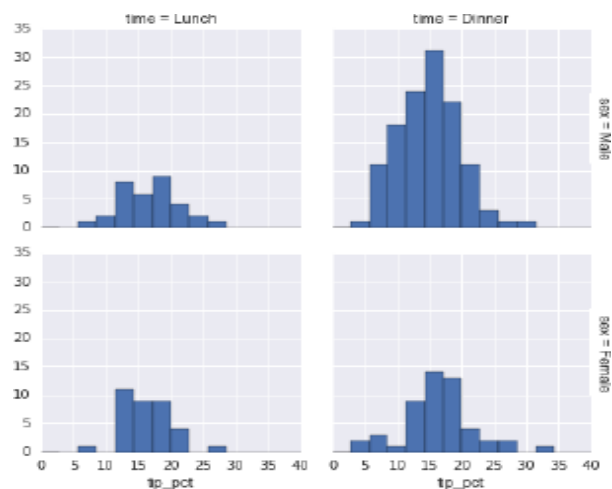
We'll demo this with the Iris dataset, which lists measurements of petals and sepals of three iris species:

```
import seaborn as sns
iris = sns.load_dataset("iris")
sns.pairplot(iris, hue='species', size=2.5);
```



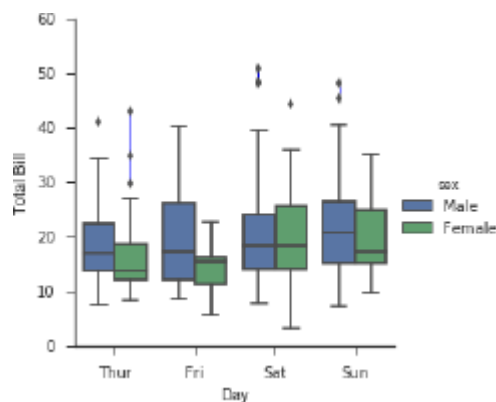
Faceted histograms

- Sometimes the best way to view data is via histograms of subsets. Seaborn's FacetGrid makes this extremely simple.
- We'll take a look at some data that shows the amount that restaurant staff receive in tips based on various indicator data



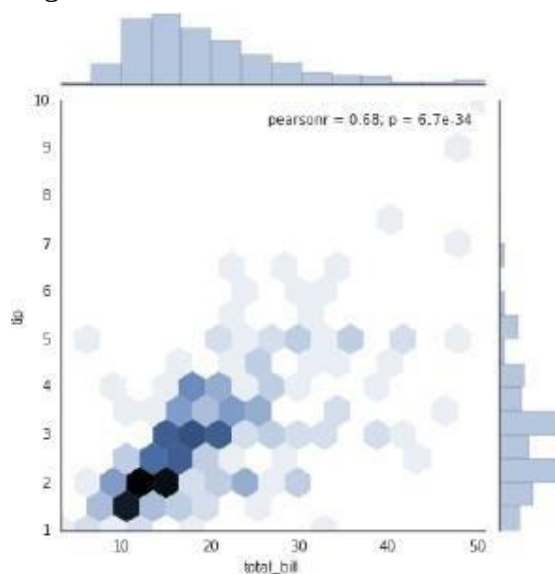
Factor plots

Factor plots can be useful for this kind of visualization as well. This allows you to view the distribution of a parameter within bins defined by any other parameter.



Joint distributions

Similar to the pair plot we saw earlier, we can use `sns.jointplot` to show the joint distribution between different datasets, along with the associated marginal distributions.



Bar plots

Time series can be plotted with `sns.factorplot`.

